An Analytical Performance Model for Streaming Tensor Programs

1 Overview

We present an analytical timing model for dataflow graphs expressed in the Streaming Tensor Programs (STeP) abstraction. The model captures the steady-state throughput and startup latency of each operator in a STeP graph, accounting for compute-bound and memory-bound regimes via Roofline modeling. Dynamic tensor dimensions propagate as symbolic variables, enabling closed-form analysis that can be instantiated with concrete traces, distributions, or worst-case bounds.

The model structure is inspired by the Stream-HLS performance model [1], adapted to STeP's operator-based dataflow semantics. The key reformulation replaces loop-nest analysis with a rate-based abstraction built on chunk intervals and tile counts, which naturally captures the multi-rate behavior of STeP operators.

2 Graph Structure

A STeP program is modeled as a directed acyclic graph (DAG) G. Each STeP operator is a node; each stream connecting a producer to a consumer is an edge. We process nodes in topological order. The total execution time of the program is:

$$T_{\text{graph}} = \max_{n \in \text{leaf}(G)} \big(end(n)\big) \tag{1}$$

3 Per-Node Quantities

For each node n in the graph, we define the following quantities, derived from the operator type, its stream shapes, and the hardware parameters.

Symbol	Name	Description
$T_{\text{fire}}(n)$	Firing time	Processing time per input step (Roofline)
$N_{\text{fire}}(n)$	Output chunk count	Total output chunks produced; may be symbolic
$NIT_n^{n'}$	Input tile count	Input tiles consumed from predecessor n' per output chunk
$OTPC(n)$	Output tiles per chunk	Output tiles produced per output chunk

Table 1: Per-node quantities defined by operator type and stream shapes.

The key distinction is that T_{fire} is the cost of one *input step*, while $NIT_n^{n'}$ counts how many tiles are consumed from predecessor n' per output chunk. What constitutes a "step" depends on the operator: BinaryMap consumes one tile from each of its two inputs simultaneously per step; FlatReassemble consumes from one selected data input per step based on a control stream. For element-wise operators ($NIT = 1$), one step produces one output. For reducing operators like Accum ($NIT = K$), each output chunk requires K sequential steps, each costing T_{fire}, giving an output chunk interval of $NIT \cdot \max(T_{\text{fire}},\ OTI(\text{pred}))$.

3.1 Firing Time (Roofline Model)

The firing time captures the cost of one input step. For compute operators, we use a Roofline model that accounts for compute cycles, PMU (scratchpad) store cycles, and PMU read cycles:

$$T_{\text{fire}}(n) = \max\left(T_{\text{PMU-read}}(n),\ \left\lceil \frac{FLOPs}{BW_c} \right\rceil,\ T_{\text{PMU-store}}(n)\right) \tag{2}$$

where BW_c is the allocated compute bandwidth (FLOPs/cycle).

PMU store cycles. When a compute operator writes its output tile to the on-chip scratchpad (PMU) — indicated by the `write_back_mu` flag — the store cost is:

$$T_{\text{PMU-store}}(n) = \left\lceil \frac{S_{\text{out}}}{BW_{\text{PMU}}} \right\rceil \tag{3}$$

where S_{out} is the output tile size in bytes and $BW_{\text{PMU}} = 64$ bytes/cycle. When the output streams directly to a FIFO (`write_back_mu` = false), $T_{\text{PMU-store}} = 0$.

PMU read cycles. When a compute operator's inputs come from PMU-resident tiles (produced by off-chip loads, Bufferize, or other PMU-writing operators), the read cost is charged:

$$T_{\text{PMU-read}}(n) = \sum_{\substack{n' \in \text{pred}(n) \\ n' \text{ produces PMU tile}}} \left\lceil \frac{S_{n'}}{BW_{\text{PMU}}} \right\rceil \tag{4}$$

where $S_{n'}$ is the tile size of predecessor n'. Pass-through operators (no compute, no `write_back_mu`) do not incur PMU read costs. Predecessors whose output lives in streaming FIFOs (not PMU) contribute zero read cost.

Structural operators. For structural operators with no compute (shape operators, routing operators, Bufferize), $T_{\text{fire}}(n) = 0$.

FLOPs estimation. The per-firing FLOPs depend on the operator's function:

- **Matmul**: $2 \cdot M \cdot K \cdot N$ for tile shapes $[M, K] \times [K, N]$.
- **RowWiseSum**: $R \cdot C$ for tile shape $[R, C]$.
- **Default** (element-wise): $R \cdot C$ for output tile shape $[R, C]$, or 1 for scalar.
- **Accum**: $R \cdot C$ per accumulation step (one FMA per input element).

4 Core Timing Equations

We compute the following quantities for each node n in topological order.

4.1 Start Time

A node cannot begin execution until all its input streams carry data. For source nodes (no predecessors), $st(n) = 0$.

$$st(n) = \max_{n' \in \text{pred}(n)} \big(fto(n')\big) \tag{5}$$

A consumer can start as soon as each predecessor emits its first output tile, so the start time is gated by the slowest predecessor's first-tile-out time.

4.2 First Tile Out

The absolute time at which node n produces its first output tile. Before producing output, the node must (1) wait for enough input tiles to arrive to complete its first firing, and (2) perform one firing of computation.

$$fto(n) = st(n) + ICD(n) + T_{\text{eff}}(n) \tag{6}$$

where $T_{\text{eff}}(n)$ is the effective single-firing latency:

$$T_{\text{eff}}(n) = \begin{cases} OTI_{\text{HBM}}(n) & \text{if } n \text{ is an off-chip memory op} \\ T_{\text{fire}}(n) & \text{otherwise} \end{cases} \tag{7}$$

For off-chip operators, the first-tile latency is the HBM access time rather than compute time, since these operators have no compute ($T_{\text{fire}} = 0$) and the data rate is decoupled from the latency of sending data to/from HBM.

Input Chunk Delay. $ICD(n)$ is the time from when the first input tile arrives until the full input chunk for the first firing is available. It captures the wait for the remaining $NIT - 1$ tiles at each predecessor's effective output rate:

$$ICD(n) = \max_{n' \in \text{pred}(n)} \left((NIT_n^{n'} - 1) \cdot \max(T_{\text{fire}}(n),\ OTI(n')) \right) \tag{8}$$

The $\max(T_{\text{fire}}, OTI(n'))$ term reflects that each input tile consumption is gated by both the predecessor's production rate and the consumer's own processing time.

4.3 End Time

The absolute time at which node n completes all its firings:

$$end(n) = fto(n) + \left(\left\lceil \frac{N_{\text{fire}}(n)}{OTPC(n)} \right\rceil - 1 \right) \cdot OCI(n) \tag{9}$$

This counts from the first tile out: after the first firing completes, the remaining $\lceil N_{\text{fire}}/OTPC \rceil - 1$ output chunks arrive at steady-state intervals of OCI.

4.4 Output Chunk Interval (OCI)

The time between consecutive output chunks produced by node n.

Compute operators. The OCI is computed via a unified formula that folds the firing time into each predecessor's contribution:

$$OCI(n) = \max_{n' \in \text{pred}(n)} \left(NIT_n^{n'} \cdot \max(T_{\text{fire}}(n),\ OTI(n')) \right) \tag{10}$$

Each predecessor contributes NIT sequential steps, each gated by $\max(T_{\text{fire}}, OTI(n'))$. The output rate is limited by the slowest predecessor. Since $NIT \geq 1$, the result is always $\geq T_{\text{fire}}$. For non-accumulating operators where $NIT = 1$ for all predecessors, this reduces to $\max_{n'}(\max(T_{\text{fire}}, OTI(n')))$.

Off-chip memory operators. The OCI is determined by the HBM contention model (Section 5) and input availability:

$$OCI(n) = \max\left(OTI_{\text{HBM}}(n),\ \max_{n' \in \text{pred}(n)} \left(NIT_n^{n'} \cdot OTI(n') \right) \right) \tag{11}$$

4.5 Output Tile Interval (OTI)

The time between consecutive output tiles produced by node n:

$$OTI(n) = \frac{OCI(n)}{OTPC(n)} \tag{12}$$

Reshape with padding. When a Reshape operator has a padding function, padding tiles are generated locally at zero cost (no waiting for predecessor). The effective per-tile OTI is:

$$OTI_{\text{reshape-pad}}(n) = \frac{OCI(n)}{chunk\_size(n)} \tag{13}$$

which distributes the OCI across all tiles in the chunk (both real and padding), since the padding tiles are "free."

FlatPartition fan-out. Tiles from a FlatPartition are distributed round-robin to N_c consumers. Each consumer sees tiles at N_c times the interval:

$$OTI_{\text{FlatPart}}(n) = OTI(n) \cdot N_c \tag{14}$$

FlatReassemble fan-in. FlatReassemble collects tiles from N_d parallel data inputs. Since expert outputs arrive interleaved (round-robin), the effective per-predecessor OTI seen by the reassembler is:

$$OTI_{\text{eff}}(n') = \frac{OTI(n')}{N_d} \tag{15}$$

This divisor applies when computing *ICI* and *ICD* at a FlatReassemble node.

4.6 Summary of Dependencies

The equations form a chain that propagates rates through the graph in topological order. For each node, the computation order is:

$$OTI(\text{predecessors}) \;\rightarrow\; OCI(n) \;\rightarrow\; OTI(n)$$

In parallel, the startup path is:

$$OTI(\text{predecessors}) \;\rightarrow\; ICD(n) \;\rightarrow\; fto(n) \;\rightarrow\; st(\text{successors})$$

The total graph latency combines both: a startup transient (accumulated through the fto/st chain) plus a steady-state execution phase ($(\lceil N_{\text{fire}}/OTPC \rceil - 1) \cdot OCI$ at each node).

5 Memory Contention Model

When multiple off-chip memory operators are active concurrently, they compete for a shared set of HBM channels. We model this with a two-pass approach and a detailed per-tile HBM access model.

5.1 Per-Tile HBM Access Model

For each off-chip operator n, the per-tile HBM access time models the pipelined dispatch and channel processing:

$$OTI_{\text{HBM}}(n) = \max(1,\ \text{bottleneck} + T_{\text{startup}} + T_{\text{sim}}) \tag{16}$$

where:

$$\text{dispatch} = \left\lceil \frac{R}{P} \right\rceil \qquad \text{(address dispatch cycles)} \tag{17}$$

$$\text{channel\_pipe} = \left(\left\lceil \frac{R_{\text{concurrent}}}{C} \right\rceil - 1 \right) \cdot I_{\text{init}} \qquad \text{(busiest channel pipeline)} \tag{18}$$

$$\text{bottleneck} = \max(\text{dispatch}, \text{channel\_pipe}) + L_{\text{ch}} \qquad \text{(pipelined overlap)} \tag{19}$$

Here R is the number of HBM requests per tile ($= \lceil \textit{tile\_bytes}/\textit{hbm\_addr\_offset} \rceil$), P is the parallel dispatch width (`par_dispatch`), C is the number of HBM channels, I_{init} is the per-channel initiation interval, L_{ch} is the per-channel response latency, T_{startup} is the per-tile channel startup overhead, and $T_{\text{sim}} = 3$ cycles accounts for per-tile bookkeeping overhead in the simulator's blocking per-tile loop (three `time.tick()` calls per tile in `linear_offchip_load.rs`: send-request timestamp, read-finish timestamp, and on-chip enqueue timestamp).

$R_{\text{concurrent}}$ is the total number of concurrent HBM requests from all overlapping off-chip operators (see below).

5.2 Two-Pass Contention Estimation

Precisely identifying which off-chip operators overlap requires knowing their time intervals, which depend on OTI_{HBM}. We resolve this with two passes:

Pass 1 (no contention). Compute OTI_{HBM} for each off-chip operator assuming it is alone ($R_{\text{concurrent}} = R$). Run a full timing pass to estimate $[st, end]$ intervals for all nodes.

Pass 2 (with contention). For each off-chip operator n, compute the rate-weighted concurrent requests from all off-chip operators whose $[st, end]$ intervals (from Pass 1) overlap with n's interval:

$$R_{\text{concurrent}}(n) = \max\left(R_n, \left\lceil \sum_{\substack{n' \in \text{offchip} \\ [st_{n'}, end_{n'}] \cap [st_n, end_n] \neq \emptyset}} R_{n'} \cdot f_{\text{overlap}}(n, n') \cdot f_{\text{rate}}(n, n') \right\rceil \right) \tag{20}$$

where f_{overlap} is the fractional temporal overlap and $f_{\text{rate}} = \min(1, OCI_n / OCI_{n'})$ scales by relative firing rate (a slow operator with high OCI injects fewer requests per target tile access). Recompute OTI_{HBM} with the updated $R_{\text{concurrent}}$ and run a second timing pass.

6 Per-Operator Parameters

The tables below define T_{fire}, N_{fire}, NIT, and $OTPC$ for each STeP operator category. Stream shapes are written as $[D_a, \ldots, D_0]$. Symbolic variables (e.g. D_i) propagate through the model.

6.1 Shape and Pass-Through Operators

This category includes Flatten, Promote, PromoteOuter, Reshape (without padding), Expand, ExpandRef, RepeatRef, RepeatStatic, and Broadcast. These act as zero-cost pass-through for timing purposes.

Quantity	Value
T_{fire}	0
N_{fire}	output stream volume
NIT	1 per input
$OTPC$	1

Note: ExpandRef and RepeatRef take two inputs (data + reference); both have $NIT = 1$ per input.

6.2 UnaryMap

Element-wise computation with a single input. One input tile produces one output tile.

Quantity	Value
T_{fire}	$\max(\lceil F/BW_c \rceil,\ T_{\text{PMU-store}})$ (Eq. 2)
N_{fire}	$\prod D_i$
$NIT_n^{n'}$	1
$OTPC$	1

6.3 BinaryMap

Element-wise computation with two inputs consumed in lockstep.

Quantity	Value
T_{fire}	$\max(\lceil F/BW_c \rceil,\ T_{\text{PMU-store}})$ (Eq. 2)
N_{fire}	$\prod D_i$
$NIT_n^{n'}$	1 (for each of the two input streams)
$OTPC$	1

6.4 Accum

Reduction over the inner b dimensions. Input shape $[D_a, \ldots, D_b, D_{b-1}, \ldots, D_0]$. Let $K = \prod_{i=0}^{b-1} D_i$ (elements per reduction) and $M = \prod_{i=b}^{a} D_i$ (number of outputs). K may be symbolic.

Quantity	Value
T_{fire}	$\max(\lceil F/BW_c \rceil,\ T_{\text{PMU-store}})$ ($F = R \cdot C$ per accumulation step)
N_{fire}	M
$NIT_n^{n'}$	K (consumes K input tiles per output)
$OTPC$	1

6.5 BinaryMapAccum (Scan)

Two-input reduction: identical to Accum but with two input streams. Each firing consumes K tiles from both inputs. Internal accumulator state does not affect the production rate.

6.6 Bufferize

Accumulates $K = \prod_{i=0}^{b-1} D_i$ input tiles, emits one buffer reference. Scratchpad writes are pipelined with input arrival.

6.7 Streamify

Reads tiles from an on-chip buffer. Let R be the number of buffer references and E be the number of tiles read per buffer (including repeat factors).

Quantity	Value
T_{fire}	$\max(\lceil F/BW_c \rceil,\ T_{\text{PMU-store}})$
N_{fire}	M (output stream volume)
$NIT_n^{n'}$	K (for each of the two input streams)
$OTPC$	1

Quantity	Value
T_{fire}	0
N_{fire}	$\prod_{i=b}^{a} D_i$ (number of buffers emitted)
$NIT_n^{n'}$	K (tiles accumulated per buffer)
$OTPC$	1

6.8 DynStreamify

Reference-addressed variant of Streamify. Takes a buffer input and a reference stream that controls read addresses.

6.9 Off-Chip Memory Operators

This category includes LinearOffChipLoad, LinearOffChipLoadRef, RandomOffChipLoad, DynLinearOffChipLoad, RandomOffChipStore, DynOffChipStore, and LinearOffChipStore. The OTI is determined by the HBM contention model (Section 5) rather than the Roofline equation.

Source loads (LinearOffChipLoad, DynLinearOffChipLoad) have no predecessors in the graph ($NIT = \{\}$). Reference-addressed loads (LinearOffChipLoadRef, RandomOffChipLoad) consume one reference tile per firing. Stores consume one data tile per firing; RandomOffChipStore additionally consumes one address tile.

6.10 Parallelize

Splits a stream across N_c parallel consumers by dividing the outermost dimension.

6.11 FlatPartition and FlatReassemble

Control-driven routing for data-dependent parallelism (e.g., Mixture-of-Experts).

FlatPartition. Routes input tiles to N_c consumers based on a control stream. $T_{\text{fire}} = 0$, $OTPC = 1$. Consumes one data tile and one control tile per firing ($NIT = 1$ for each). N_{fire} is the sum of output stream volumes across all consumers. The OTI is multiplied by N_c (fan-out effect) so each consumer sees tiles at $N_c\times$ the base interval.

FlatReassemble. Collects tiles from N_d data inputs under control-stream guidance. $T_{\text{fire}} = 0$, $OTPC = 1$, $NIT = 1$ per input (data + control). The fan-in divisor N_d is applied to predecessor OTI values when computing ICI and ICD.

6.12 EagerMerge

Nondeterministic (arrival-order) merging of multiple input streams. $T_{\text{fire}} = 0$, $NIT = 1$ per input, $OTPC = 1$. The model treats it identically to other routing operators, with $st(n)$ determined by the first branch to produce data and $end(n)$ by the last.

Quantity	Value
T_{fire}	E (1 cycle per output tile)
N_{fire}	R (number of buffer references consumed)
$NIT_n^{n'}$	1 (one buffer reference per firing)
$OTPC$	E (tiles emitted per buffer read)

Quantity	Value
T_{fire}	$\lceil S_{\text{tile}} \cdot 8 / BW_{\text{scratchpad}} \rceil$ (bits / scratchpad bandwidth)
N_{fire}	ref stream volume
$NIT_n^{n'}$	1 per input (buffer + reference)
$OTPC$	E (tiles per buffer)

6.13 Reshape (with Padding)

When Reshape has a padding function (`pad_fn`), it groups input tiles into chunks of `chunk_size`, padding incomplete chunks. $T_{\text{fire}} = 0$, $NIT = 1$, $OTPC = 1$. The OTI adjustment for padding is described in Section 4.5.

6.14 RetileStreamify and FlatmapFilterRowStreamify

Specialized FlatMap variants. RetileStreamify splits a tile into `num_chunks` sub-tiles: $T_{\text{fire}} = 0$, $OTPC = num\_chunks$. FlatmapFilterRowStreamify filters rows based on a mask: $T_{\text{fire}} = 0$, $NIT = 1$ per input (data + mask), $OTPC = R$ (tile rows).

7 Handling Dynamic Dimensions

STeP streams may have dynamic-regular or ragged dimensions. These appear as symbolic variables throughout the model (e.g., D_i for the number of tokens routed to expert i, or K for a dynamic reduction dimension in Accum).

7.1 Expected-Value Substitution

For FlatPartition and FlatReassemble operators with dynamic dimensions (`DynDim`), the model computes expected-value substitutions assuming uniform routing:

- **FlatPartition**: Expected tiles per consumer = $\lfloor N_{\text{fire}}(\text{input})/N_c \rfloor$.
- **FlatReassemble**: Expected elements per control firing = $\lfloor \sum N_{\text{fire}}(\text{data inputs})/N_{\text{fire}}(\text{control}) \rfloor$.

These substitutions are applied before the timing passes so that DynDim symbols are resolved early in rate computations.

7.2 Alternative Evaluation Strategies

The timing equations remain closed-form expressions over symbolic variables. Beyond expected-value substitution, two other strategies are available:

Concrete instantiation. Substitute symbols with values from a specific trace.

Worst/best-case bounds. Substitute extreme values to obtain performance envelopes.

8 Relationship to the Stream-HLS Model

The model shares the same DAG-based, topological-order structure as Stream-HLS [1]. The key differences are:

Quantity	Value
T_{fire}	0 (rate set by HBM model, not Roofline)
N_{fire}	output/input stream volume
$NIT_n^{n'}$	1 per input (address/data/reference streams)
$OTPC$	1

Quantity	Value
T_{fire}	0
N_{fire}	$\sum_{i=1}^{N_c} \prod D_j^{(i)}$ (total across all output streams)
$NIT_n^{n'}$	1
$OTPC$	1

(i) **Rate-based reformulation with folded T_fire.** Stream-HLS tracks FW_n, LW_n, and $LR_n^{n'}$ as absolute relative times, with a Depend/Epilogue decomposition to handle stalls. This model uses a unified OCI formula (Eq. 10) where T_{fire} is folded into each predecessor's contribution via $NIT \cdot \max(T_{\text{fire}}, OTI(n'))$, naturally capturing both backpressure and compute-bound behavior.

(ii) **Uniform streaming edges.** Stream-HLS distinguishes FIFO edges ($WAF = RAF$) from shared-buffer edges. In this model, all edges are streaming. Buffering delays emerge from Bufferize nodes (which have $NIT = K$, causing a proportional ICD at downstream consumers) rather than from edge classification.

(iii) **Three-term Roofline with PMU modeling.** Per-node timing uses a Roofline model that separately accounts for PMU reads, compute, and PMU stores, with the PMU store term gated by `write_back_mu`.

(iv) **Detailed HBM contention model.** Off-chip memory operators use a physics-based per-tile HBM model (dispatch, channel pipelining, startup, sim overhead) with a two-pass temporal-overlap contention estimation, rather than a simple R_{total}/C approximation.

(v) **Symbolic dimensions with expected-value resolution.** Node parameters may contain symbolic variables representing data-dependent stream dimensions. FlatPartition/FlatReassemble DynDim symbols are resolved via uniform-routing expected values before timing computation.

References

[1] S. Basalama and J. Cong, "Stream-HLS: Towards Automatic Dataflow Acceleration," in *Proc. ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25)*, Monterey, CA, USA, Feb. 2025, pp. 103–114.

[2] G. Sohn, G. Zhang, K. Hossfeld, J. Kim, N. Sobotka, N. Zhang, O. Hsu, and K. Olukotun, "Streaming Tensor Programs: A Streaming Abstraction for Dynamic Parallelism," in *Proc. 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, Pittsburgh, PA, USA, Mar. 2026.